Comparison of Transfer Learning and Random Initialization for Image Classification

Shriman Raghav Srinivasan
Individual Project
Northeastern University, CS 5330 Final Project Proposal
Email: srinivasan.shri@northeastern.edu

***Abstract*—Fine-tuning an ImageNet-pretrained network usually beats training the same network from scratch, but freezing the pretrained layers changes two things at once: the quality of the features, and the number of parameters left to train. This proposal separates those two effects. On a six-class scene dataset I compare a pretrained network with its early layers frozen, the same architecture with those early layers randomly initialized and frozen, and the whole network trained from random weights. The middle condition keeps the trainable parameter count identical to the pretrained one, so any gap between them reflects feature quality rather than capacity. I also sweep the freeze boundary from a linear probe up to full fine-tuning, and read the differences through per-epoch learning curves and a projection of the learned features. Every condition uses validation-based early stopping across three random seeds, with the test split left untouched until the final numbers.**

I. Introduction

A network trained from random weights usually needs a large labeled dataset to reach good accuracy without overfitting, and it converges slowly. Transfer learning avoids both problems: the network starts from weights learned on a large source dataset such as ImageNet [2], then adapts to a smaller target task. Frozen pretrained features are already useful on their own. A linear classifier placed on top of them is a well-known strong baseline [5], [6], and better source models tend to transfer better [7]. He et al. [4] pushed back on the idea that pretraining is always necessary, showing that from-scratch training can catch up given enough data and iterations, which frames the benefit of pretraining as mostly one of speed and low-data efficiency.

The problem is that the obvious experiment measures two things at the same time. When a pretrained network with frozen early layers beats a network trained from scratch, the win could come from the quality of the pretrained features or simply from having fewer parameters to fit. I separate the two with a three-condition design, and ask how much of the benefit survives once trainable capacity is held fixed.

II. Problem Formulation

Let f_θ be a convolutional network with parameters θ, split into a frozen part θ_f and a trainable part θ_t, and let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$ be the target training set. Every condition minimizes the same cross-entropy loss over θ_t,

$$\mathcal{L}(\theta_t) = -\frac{1}{N}\sum_{i=1}^{N} \log p_\theta(y_i \mid x_i), \tag{1}$$

and they differ only in how θ_f and θ_t start out: (i) *pretrained-frozen*, with θ_f taken from ImageNet weights [1], [2] and held fixed; (ii) *random-frozen*, the same split but with θ_f random and fixed, which matches the trainable capacity of (i); and (iii) *random-full*, where nothing is frozen. Condition (i) against (ii) measures feature quality at fixed capacity; (ii) against (iii) measures capacity on its own.

III. Proposed Approach

The backbone is ResNet-18 [1]. I make θ_t the last convolutional stage (`layer4`) and a fresh six-class head, and freeze the earlier stages, since early layers tend to learn generic edge- and color-like features while later ones specialize [3]. One detail matters for the frozen conditions: the batch-normalization [8] layers inside θ_f are kept in evaluation mode during training so their running statistics do not shift. Left in training mode they would quietly corrupt the frozen representation. Inputs are resized so the shorter side is 224px, center-cropped to 224×224, and normalized with the usual ImageNet channel statistics, the same for every condition. The data is the Intel Image Classification set [11] (buildings, forest, glacier, mountain, sea, street). It ships with only train and test folders, so I carve a stratified 10% validation split out of the training data and keep the test set aside for the final numbers alone. Each condition runs on three seeds and stops when validation loss stops improving, rather than after a fixed number of epochs.

I also want to know whether `layer4` is the right place to cut. The plan therefore includes a freeze-depth sweep on the pretrained model: start from a linear probe that trains only the head, then unfreeze `layer4`, `layer3`, `layer2`, and finally everything, and plot accuracy against how much of the network is trainable.

IV. Evaluation Plan

All reporting happens on the held-out test split, after model selection is done. For each condition I report mean and standard deviation of test accuracy across seeds, per-class precision, recall and F1, and a confusion matrix from the seed nearest the mean. I expect those matrices to show the usual look-alike pairs (glacier with mountain, buildings with street) whatever the condition. Convergence speed is part of the story, so I also report epochs to early stopping. Two extra views back up the accuracy numbers: learning curves that show how

differently the conditions converge, and a PCA of the last-layer features that shows whether the pretrained and random-frozen representations actually pull the classes apart. Where a fair comparison exists, such as published Intel-dataset baselines or the transfer study of Kornblith et al. [7], I compare against it.

V. Feasibility

The two frozen conditions train very few parameters and are cheap; random-full trains more but is still small at this dataset size. The three conditions across three seeds, plus the freeze-depth sweep, should finish comfortably on a single consumer GPU. My expectation, and the intended result rather than a failure case, is that random-full converges more slowly and generalizes a little worse than the pretrained model, and that the gap between the two frozen conditions pins down what pretraining specifically contributes. The random-frozen baseline should land well above chance despite its random features, in line with the finding that even random convolutional filters keep some useful structure [9]. The confusion between similar scenes is a property of the data, familiar from scene-recognition work on the Places database [10], not of any one training setup, so it should look much the same across all three conditions.

References

[1] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[2] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A Large-Scale Hierarchical Image Database," in *IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2009, pp. 248–255.

[3] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, "How Transferable Are Features in Deep Neural Networks?" in *Advances in Neural Information Processing Systems (NeurIPS)*, 2014, pp. 3320–3328.

[4] K. He, R. Girshick, and P. Dollár, "Rethinking ImageNet Pre-training," in *IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2019, pp. 4918–4927.

[5] J. Donahue, Y. Jia, O. Vinyals, J. Hoffman, N. Zhang, E. Tzeng, and T. Darrell, "DeCAF: A Deep Convolutional Activation Feature for Generic Visual Recognition," in *Int. Conf. Machine Learning (ICML)*, 2014, pp. 647–655.

[6] A. Sharif Razavian, H. Azizpour, J. Sullivan, and S. Carlsson, "CNN Features off-the-shelf: an Astounding Baseline for Recognition," in *IEEE Conf. Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2014, pp. 806–813.

[7] S. Kornblith, J. Shlens, and Q. V. Le, "Do Better ImageNet Models Transfer Better?" in *IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 2661–2671.

[8] S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," in *Int. Conf. Machine Learning (ICML)*, 2015, pp. 448–456.

[9] A. M. Saxe, P. W. Koh, Z. Chen, M. Bhand, B. Suresh, and A. Y. Ng, "On Random Weights and Unsupervised Feature Learning," in *Int. Conf. Machine Learning (ICML)*, 2011, pp. 1089–1096.

[10] B. Zhou, A. Lapedriza, J. Xiao, A. Torralba, and A. Oliva, "Learning Deep Features for Scene Recognition using Places Database," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2014, pp. 487–495.

[11] Intel Corporation and P. Bansal, "Intel Image Classification," Kaggle, 2019. [Online]. Available: https://www.kaggle.com/datasets/puneet6060/intel-image-classification